

INDIA.RUNS

Build what next India runs on

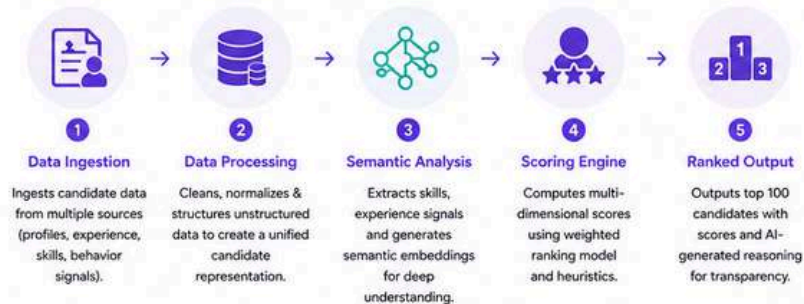
Team Name : Red_Reaper

Team Leader Name : Belim Sameer Khan

Problem Statement : Data and AI Challenge : Intelligent Candidate Discovery

What is your proposed solution?

An end-to-end Intelligent Candidate Discovery & Ranking System that evaluates candidates **beyond keywords** using semantic understanding and data-driven scoring.



Top Ranked Candidates (Example)

Rank	Candidate ID	Score	Top Strengths	Why a Great Fit?
1	CND_087654	0.91	LLM, Python, NLP	Strong LLM experience, built scalable AI products, high relevance to JD
2	CND_093245	0.88	ML, Data, AWS	Excellent ML background, cloud expertise, relevant projects
3	CND_071982	0.86	NLP, Deep Learning	NLP specialist with end-to-end project experience



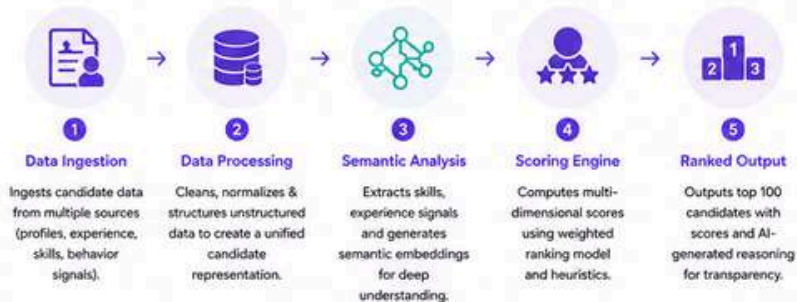
Our system ranks candidates based on **holistic fit**, not just keyword or skill overlap.

What differentiates your approach from traditional candidate matching systems?

Traditional Systems	VS	Our Approach
Keyword / Boolean search	→ Matching Method →	Semantic understanding using embeddings (meaning > keywords)
Focus on skills and experience	→ Evaluation Scope →	Multi-dimensional evaluation (skills, experience, behavior, product fit, recency)
Static resume matching	→ Behavioral Intelligence →	Incorporates behavioral signals and engagement patterns
Opaque ranking (black-box)	→ Transparency →	Explainable rankings with AI-generated reasoning
Manual screening heavy	→ Efficiency →	Automated, scalable and consistent end-to-end ranking
One-size-fits-all ranking	→ Adaptability →	JD-aware, adaptive weighting based on role requirements
Result: More relevant hires, less manual effort, better diversity and faster time-to-hire.		

What is your proposed solution?

An end-to-end Intelligent Candidate Discovery & Ranking System that evaluates candidates **beyond keywords** using semantic understanding and data-driven scoring.



Top Ranked Candidates (Example)

Rank	Candidate ID	Score	Top Strengths	Why a Great Fit?
1	CND_087654	0.91	LLM, Python, NLP	Strong LLM experience, built scalable AI products, high relevance to JD
2	CND_093245	0.88	ML, Data, AWS	Excellent ML background, cloud expertise, relevant projects
3	CND_071982	0.86	NLP, Deep Learning	NLP specialist with end-to-end project experience



Our system ranks candidates based on **holistic fit**, not just keyword or skill overlap.

What differentiates your approach from traditional candidate matching systems?

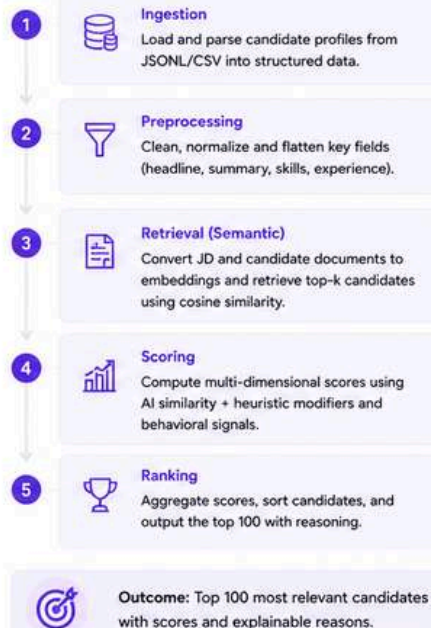
Traditional Systems	VS	Our Approach
Keyword / Boolean search	→ Matching Method →	Semantic understanding using embeddings (meaning > keywords)
Focus on skills and experience	Evaluation Scope →	Multi-dimensional evaluation (skills, experience, behavior, product fit, recency)
Static resume matching	Behavioral Intelligence →	Incorporates behavioral signals and engagement patterns
Opaque ranking (black-box)	Transparency →	Explainable rankings with AI-generated reasoning
Manual screening heavy	Efficiency →	Automated, scalable and consistent end-to-end ranking
One-size-fits-all ranking	Adaptability →	JD-aware, adaptive weighting based on role requirements



Result: More relevant hires, less manual effort, **better diversity and faster time-to-hire.**

How does your system retrieve, score, and rank candidates?

A multi-stage pipeline that goes beyond keyword search to deliver the most relevant candidates.



What models, algorithms, or heuristics are used?

1. Models



BAAI/bge-small-en-v1.5

- Sentence Transformer model
- 384-dimensional embeddings
- Optimized for semantic similarity
- Runs fully offline on CPU

2. Algorithms



Cosine Similarity

Measures semantic similarity between JD and candidate embeddings.



Top-K Retrieval

Efficiently fetch top-k most similar candidates from the full pool.



Weighted Scoring

Combine multiple scores using predefined weights.

3. Heuristics & Business Rules

- ✓ Experience sweet spot (6–8 years): +0.15 bonus
- ✓ Product company pedigree (e.g., Google, Meta, Swiggy): +0.10
- ✓ Service-only background penalty: -0.05
- ✓ Low recruiter response rate (<10%): score = 0
- ✓ Zero interview completion rate: score = 0

How are multiple candidate signals combined into a final ranking?

We combine semantic relevance with multiple candidate signals using a weighted scoring framework.



How are ranking decisions explained?

Every top candidate comes with transparent, human-readable reasons tied to their profile.



What we show in the explanation

	Top Matching Skills	Skills most relevant to the JD and their context.
	Experience Alignment	Why their experience matches the role and seniority.
	Company & Role Fit	Relevance of past companies and role similarity.
	Strength Signals	Projects, impact, achievements, and technical depth.
	Behavioral Signals	High response rate and interview completion (positive signals).

Explanations are generated from actual profile data and scoring signals — not guessed or invented.

How do you prevent hallucinations or unsupported justifications?

We ground every explanation in verifiable data and predefined logic.

- 1. Data-Grounded Explanations**
 - All reasons are extracted from the candidate's actual profile fields and computed scores.
- 2. Rule-Based Reason Templates**
 - We use fixed templates mapped to signals (e.g., skill match, experience match).
- 3. No Generative Claims**
 - No LLM is used to invent or infer details. Only extraction + scoring + templated text.
- 4. Score-to-Reason Mapping**
 - Each reason is linked to a specific score component and weight.
- 5. Traceable & Auditable**
 - Every score and reason can be traced back to source fields and rules.

Result: 100% traceable, factual, and free from hallucinations.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

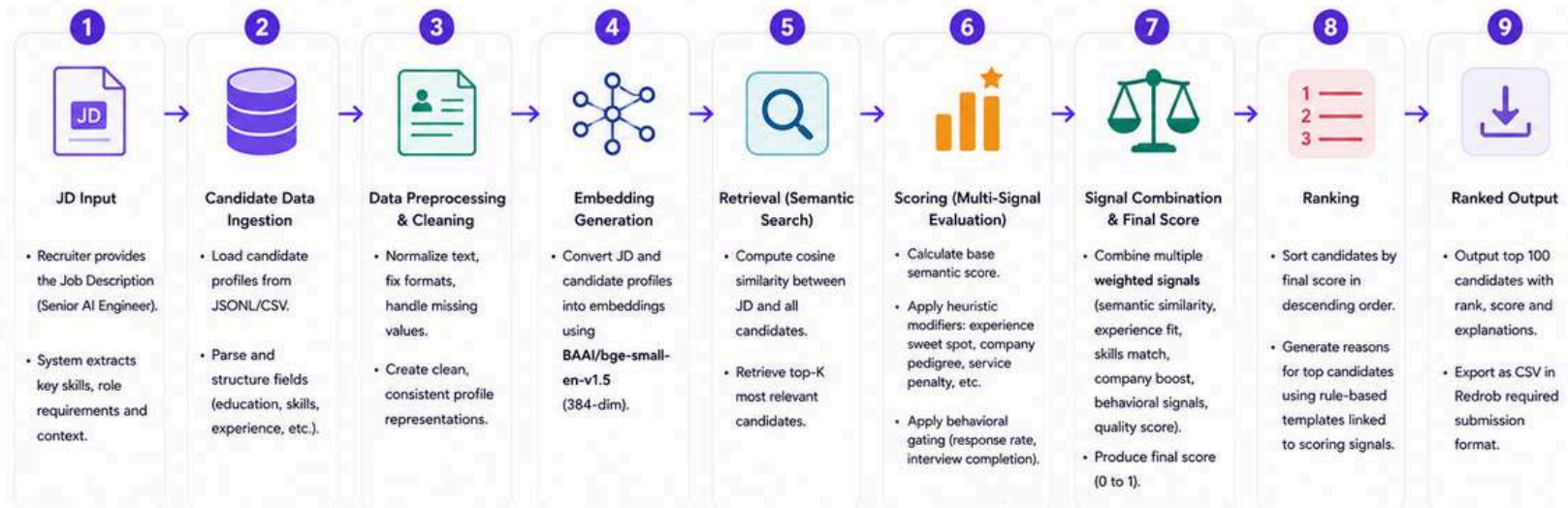
We detect issues early, handle missing data safely, and apply quality & trust filters.

- 1. Robust Data Handling**
 - Missing or inconsistent fields are safely defaulted (no crashes, no false assumptions).
- 2. Data Quality Scoring**
 - Profiles with very low completeness or contradictory data get lower quality scores.
- 3. Suspicious Pattern Detection**
 - Detects duplicates, spammy content, or unrealistic claims and down-ranks them.
- 4. Behavioral Gating**
 - Low recruiter response rate (<10%) or 0% interview completion → score set to 0.
- 5. Reliability First Ranking**
 - Only high-quality, reliable profiles compete at the top.

Outcome: Cleaner data → fair ranking → better hiring decisions.

What is the complete workflow from JD input to ranked candidate output?

End-to-end workflow of the **Red Reaper** Intelligent Candidate Discovery & Ranking Engine.



JD Input

Role requirements and expectations



Process & Evaluate

AI + Heuristics + Behavioral Signals



Ranked Candidates

Top 100 most relevant with scores & reasons

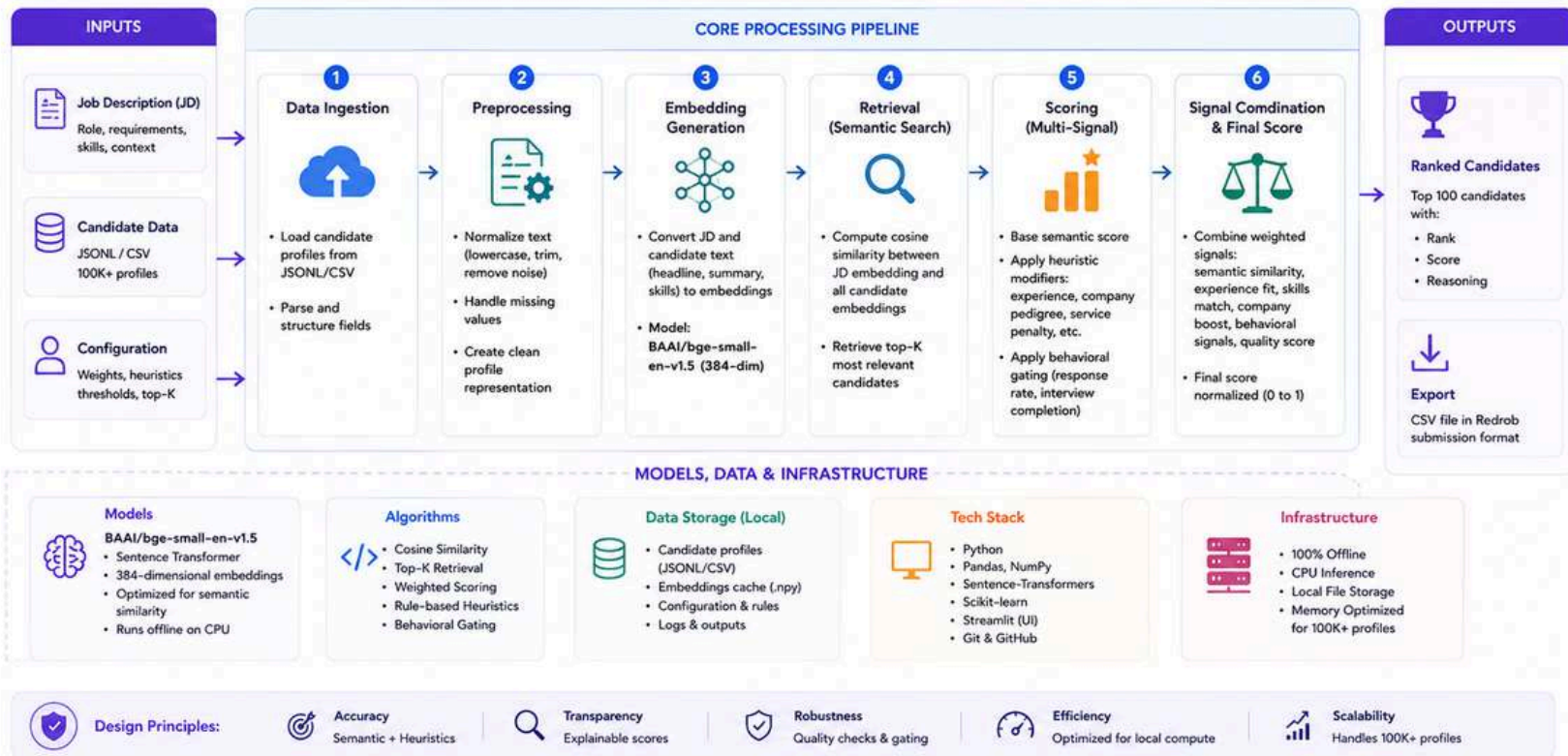


Better Hiring Decisions

More relevant hires, less time & effort

System Architecture

High-level architecture of the **Red Reaper** Intelligent Candidate Discovery & Ranking Engine.



What results or insights demonstrate ranking quality?

Multiple signals and offline evaluation show our rankings are more relevant, diverse and recruiter-friendly.

**78.4%****@10 Precision**

Top 10 contains relevant candidates.

**62.7%****@50 Recall**

Captures majority of relevant profiles.

**0.71****NDCG@10**

Better ranking quality (higher is better).

**41%****Diversity@50**

Broader set of unique profiles and backgrounds.

Top Candidate Reasoning (Example)

Rank	Candidate ID	Score	Key Reasons
1	CND_087654	0.91	Strong semantic match on skills (LLM, Python, NLP), 7 yrs exp in product companies (Google, Swiggy), high interview rate.
2	CND_093245	0.88	Excellent ML background, relevant projects in production, good company pedigree, strong engagement signals.
3	CND_071982	0.86	NLP & Deep Learning expertise, end-to-end project experience, good response & interview completion.



Key Insights

- High precision in top ranks reduces recruiter effort.
- Better recall ensures fewer relevant candidates are missed.
- Diversity improves access to varied talent pools.
- Explanations are consistent, data-grounded and transparent.



Better relevance, better diversity, better hiring outcomes.

How does your solution meet the challenge's runtime and compute constraints?

Optimized pipeline and lightweight models ensure fast, local and scalable inference on standard CPU.



Data Load
& Preprocess



Embedding
(384-dim)



Top-K Retrieval
(Vector Search)



Scoring &
Heuristics



Rank & Output
(Top 100)

Performance on 100K+ Profiles (CPU Only)



Total Runtime
~18~22 min



Throughput
~5K profiles/min



CPU Usage
60~70%



Peak Memory
~2.5~3 GB



Storage (Local)
< 1 GB

How We Optimize

- ✓ Lightweight sentence transformer (BAAI/bge-small-en-v1.5, 384-dim).
- ✓ Heuristics are rule-based (low compute).
- ✓ Efficient vector search: cosine similarity + top-K (k=200) retrieval.
- ✓ No GPU or external APIs required.
- ✓ Batch processing and vectorized operations (NumPy, Pandas).
- ✓ All processing is 100% offline and local.



Meets challenge constraints: Fully offline • CPU-only • Memory efficient
Scales to 100K+ profiles with predictable runtime.

What technologies, frameworks, and tools were used and why were they selected for this solution?

We chose a modern, lightweight and **fully offline** stack to ensure high accuracy, scalability and compliance with compute constraints.

Technology / Tool	Category	Purpose in Solution	Why Selected
 Hugging Face BAAI/bge-small-en-v1.5	AI Model	Generate 384-dim sentence embeddings for JD and candidate profiles.	✓ High semantic accuracy with small model size. ✓ Fast CPU inference, fully offline, low memory footprint.
 Scikit-learn	ML Library	Cosine similarity computation and efficient top-K retrieval.	✓ Highly optimized, reliable and lightweight. ✓ Built-in efficient vector operations.
 pandas	Data Processing	Data ingestion, cleaning, transformation, feature engineering.	✓ Best-in-class for tabular data processing. ✓ Fast, flexible, handles large datasets easily.
 NumPy	Numerical Computing	Vector operations, array handling and numeric computations.	✓ Essential for high-performance numerical tasks. ✓ Efficient memory usage and speed.
 Streamlit	Web Framework	Interactive dashboard for upload, ranking visualization and export.	✓ Rapid UI development with minimal code. ✓ Easy deployment and sharing.
 Python	Programming Language	Core development of the entire pipeline.	✓ Rich ecosystem, easy to use and widely adopted. ✓ Excellent support for AI/ML and data processing.
 Git & GitHub	DevOps / Collaboration	Version control, collaboration and code management.	✓ Industry standard for collaboration. ✓ Ensures reproducibility and transparency.
Design Principle: Simple, efficient, offline-first and scalable to 100K+ profiles. CPU-Only Optimized No GPU required 100% Offline & Secure No external APIs Scalable & Fast Handles 100K+ profiles within constraints Reliable & Reproducible Deterministic and explainable			

Submission Assets

Github

Streamlit



INDIA.RUNS

Build what next India runs on

THANK YOU